Busy-wait + Mutex

```
counter = 0     thread_count = N
barrier_mutex (unlocked)
```

```
T0----[WORK]----> LOCK ----> counter++ ----> UNLOCK ---> [while counter<N ???] ----> [CONTINUA]
                                                          ↑ spinning (consume CPU)
T1----[WORK]-----------> LOCK ----> counter++ ----> UNLOCK ---> [while counter<N ???] ----> [CONTINUA]

T2----[WORK]--------------------> LOCK ----> counter++ ----> UNLOCK ---> [while counter<N ???] ----> [CONTINUA]

T3----[WORK]------------------------------> LOCK ----> counter++ ----> UNLOCK ----------------------
--> [while counter<N ???] ----> [CONTINUA]
```

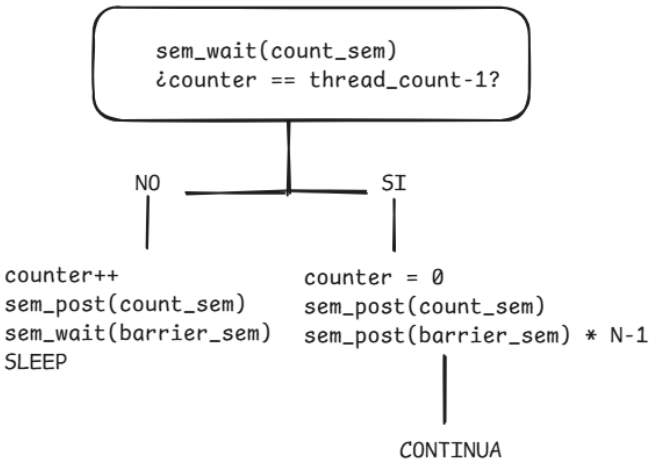
(último: counter == N, la condición se vuelve falsa y TODOS salen)

Semáforos

```
counter = 0
counter_sem = 1 (actua como mutex)
barrier_sem = 0 (bloquea a los hilos)
```

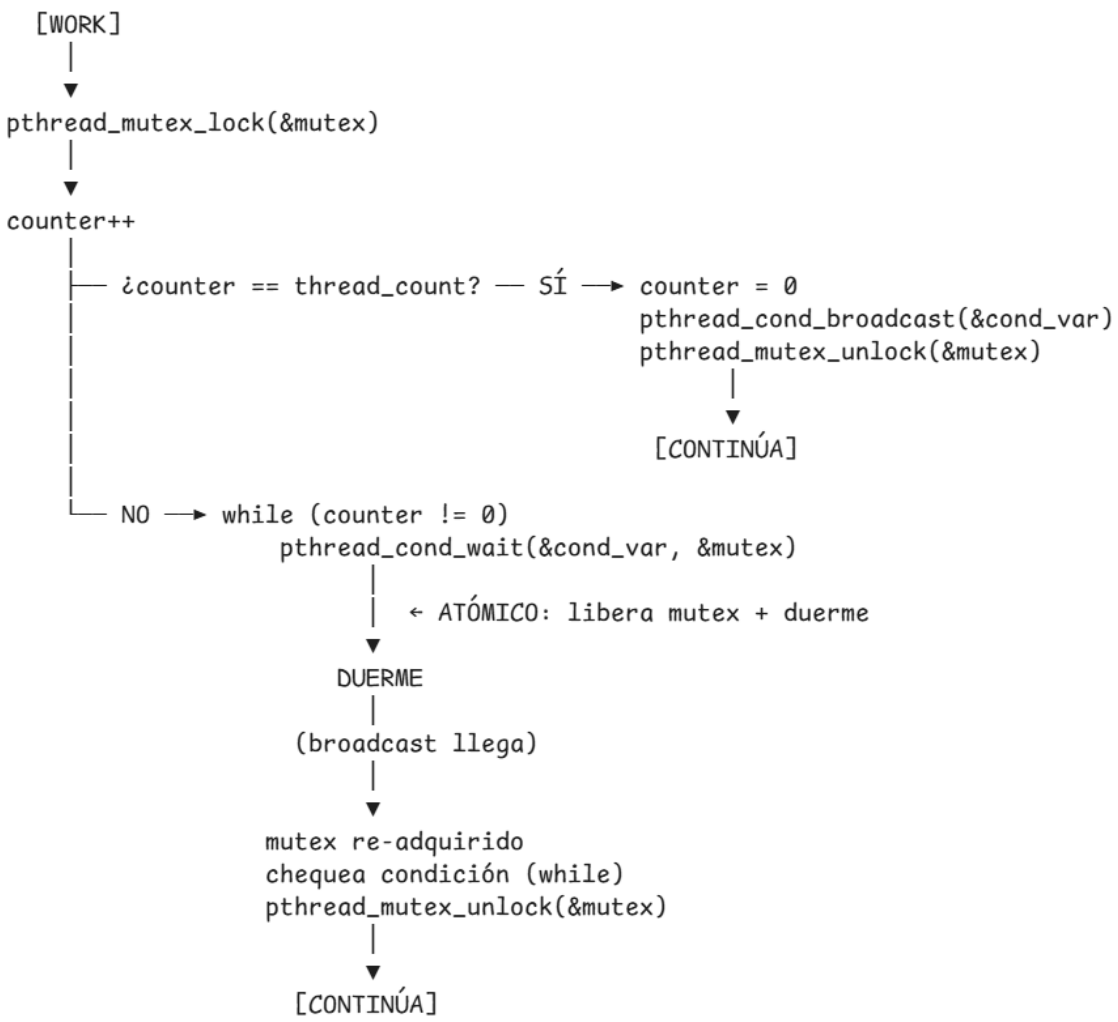
```
T0 -----[WORK]-----> sem_wait(count_sem)
                     counter++
                     sem_post(count_sem)
                     sem_wait(barrier_sem) ---> SLEEP

T3 (ULTIMO EN LLEGAR)
sem_wait(count_sem)
counter = 0 <- RESET
sem_post(count_sem)
sem_post(barrier_sem) * 3
despierta a T0, T1, T2
        ↓
[CONTINUA]

T1 -----[WORK]-----> .... igual que T0 .... ----> SLEEP

T2 -----[WORK]-----> .... igual que T0 .... -----> SLEEP
```

FLUJO DE DECISION (cada hilo):

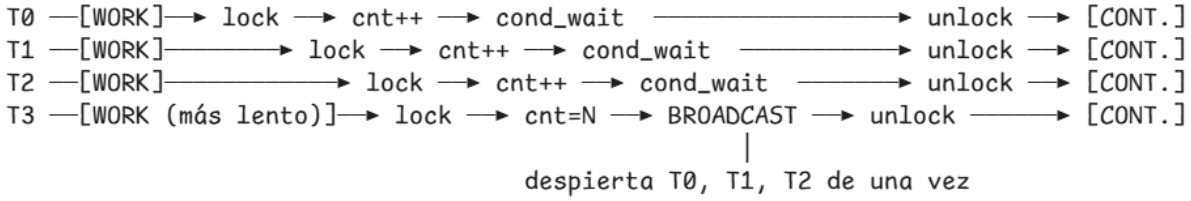


MEMORIA COMPARTIDA

CICLO DE VIDA DE CADA HILO:

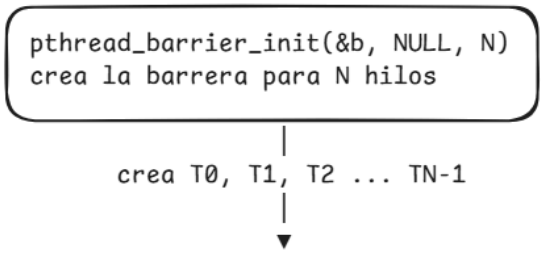


LÍNEA DE TIEMPO:

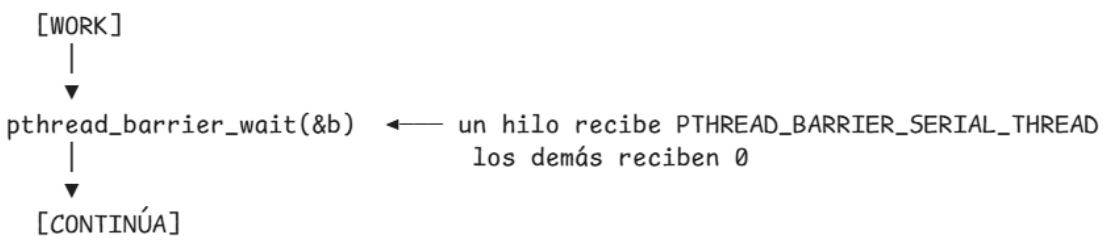


PTHREADS BARRIER (nativo)

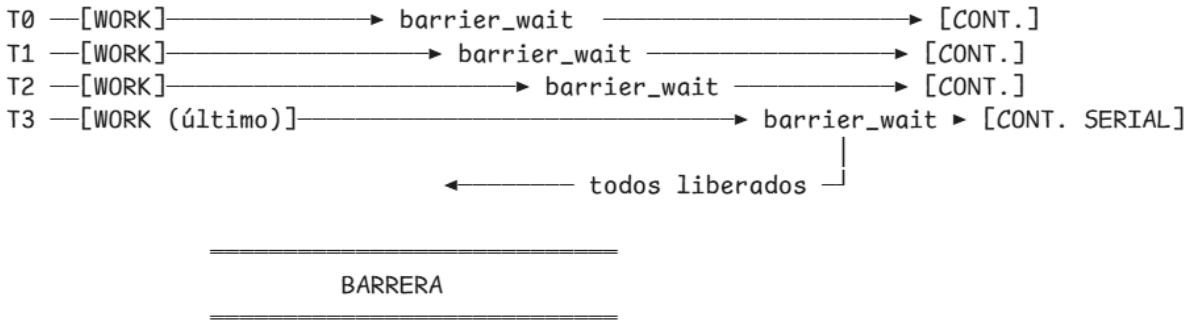
MAIN THREAD (antes de crear hilos)



CADA HILO HACE EXACTAMENTE ESTO:



LÍNEA DE TIEMPO:



CICLO DE VIDA DE LA BARRERA: